

Laboratory 1: Maze Solver

Variant 1

Group 34: Franziska Axmann and Semih Demirkan

1 Introduction

The goal of this lab is to implement a maze solver in Python. The maze is a 2D grid where 0 represents an empty cell and 1 represents a wall. Given a start and finish position, the program finds a path between them and visualizes the search process step by step. Two algorithms were implemented:

- **BFS (Breadth-First Search)** explores the maze level by level using a queue. It always finds the shortest path but uses more memory since it tracks all cells at the current distance.
- **DFS (Depth-First Search)** goes as deep as possible in one direction before backtracking, using a stack. It uses less memory but does not guarantee the shortest path.

2 Implementation

Both algorithms have the same general structure. They start by adding the start position to a data structure together with the path taken so far.

In the BFS algorithm, a queue is used (FIFO principle). It removes the element at the front of the queue, meaning it takes the cell that has been waiting the longest. In the DFS algorithm, a stack is used (LIFO principle). It removes the last element added, prioritizing the most recently discovered cell.

The `visualize()` function prints the maze after each algorithm run using the following symbols:

- **S** - start position
- **E** - finish position
- **#** - wall (cell value 1)
- ***** - cell on the final path
- **.** - explored cell (visited but not on final path)
- **[space]** - unvisited empty cell

3 Test Cases & Visualizations

To thoroughly evaluate and compare the performance of both algorithms, four different maze environments were designed as requested in the instructions.

3.1 1. Simple Small Maze

Objective: Verify basic correctness and pathfinding around simple obstacles.

Start: (1, 1) — **Finish:** (4, 5)

Input Maze Layout:

```
[1, 1, 1, 1, 1, 1, 1, 1]
[1, 0, 0, 0, 0, 0, 0, 1]
[1, 1, 1, 1, 1, 1, 0, 1]
[1, 0, 0, 0, 0, 0, 0, 1]
[1, 0, 0, 0, 0, 0, 1, 1]
[1, 1, 1, 1, 1, 1, 1, 1]
```

Output (BFS and DFS perform similarly here):

```
# # # # # # # #
# S * * * * * #
# # # # # # * #
# . . . . * * #
# . . . . * E #
# # # # # # # #
```

3.2 2. Straight Corridor (Corner Case)

Objective: Test a single-path scenario with no alternative routes.

Start: (1, 1) — **Finish:** (1, 5)

Input Maze Layout:

```
[1, 1, 1, 1, 1, 1, 1, 1]
[1, 0, 0, 0, 0, 0, 0, 1]
[1, 1, 1, 1, 1, 1, 1, 1]
```

Output (Identical for BFS and DFS):

```
# # # # # # #
# S * * * * E #
# # # # # # #
```

3.3 3. Large Open Maze

Objective: Observe performance differences in an open space.

Start: (1, 1) — **Finish:** (5, 10)

Input Maze Layout:

```
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

BFS Output (Optimal shortest path):

```
# # # # # # # # # # #
# S * * * * * * * * #
```

```

# . . . . . * #
# . . . . . * #
# . . . . . * #
# . . . . . * E #
# # # # # # # # # #

```

3.4 4. No Path Exists (Corner Case)

Objective: Verify correct failure handling when the finish is blocked.

Start: (1, 1) — **Finish:** (3, 5)

Input Maze Layout:

```

[1, 1, 1, 1, 1, 1, 1]
[1, 0, 0, 0, 0, 0, 1]
[1, 1, 1, 1, 1, 1, 1]
[1, 0, 0, 0, 0, 0, 1]
[1, 1, 1, 1, 1, 1, 1]

```

Output (Algorithm correctly stops):

```

# # # # # # #
# S . . . . #
# # # # # # #
#           #
# # # # # # #

```

4 Discussion

The results clearly show the main difference between BFS and DFS in practice. In the simple maze, BFS finds the shortest path, while DFS explores deeper and may take a less direct route. Both still reach the goal, but BFS is more efficient in terms of path optimality.

In the straight corridor, both algorithms perform almost identically since there is only one possible path. This case shows that differences between BFS and DFS only become visible in more complex mazes with branches.

In the large open maze, BFS again performs better by finding the shortest path through a systematic level-by-level search. DFS explores more unnecessary cells because it goes deep into one direction before backtracking. In the no-path case, both algorithms correctly explore all reachable cells and terminate once the search space is exhausted, confirming that both handle failure cases properly. Overall, BFS is better for shortest-path finding, while DFS is more depth-oriented and less consistent for optimal solutions.

5 Conclusion

In this lab, BFS and DFS were successfully implemented and compared for solving a maze represented as a 2D grid. The choice of data structure directly affects the behavior of an algorithm: BFS uses a queue and guarantees the shortest path, while DFS uses a stack and explores paths deeply without guaranteeing optimal results.

A key challenge was correctly managing the visited cells to avoid infinite loops and repeated work, especially in open mazes. A possible improvement would be optimizing memory usage, since storing full paths for every node can become expensive in larger mazes. Implementing more advanced algorithms like A^* could also improve efficiency while still maintaining optimal pathfinding.